FDE ACADEMY · LESSON 6 OF 9

Testing and Evaluation

Prove the system still behaves the way the business signed off on, on the bad days too.

Raunak Sharan
Enterprise Agentic AI Engineering

git checkout lesson-06

OBJECTIVES

By the end of today you can

01

Write golden cases that pin decisions, reasons and behaviour

02

Turn every fault from lessons 3–5 into a case

03

State the invariants a credit system must never break

04

Measure safety, quality, cost and latency in one run

05

Gate a release and compare against a baseline

06

Prove the harness can fail, by breaking the system

TESTS VS EVALS

Two different questions

A unit test

“Does this function work?”

- One component, isolated
- Written by engineers
- Pass or fail

An eval

“Does the system still behave as signed off?”

- The whole workflow, with faults
- Cases a credit officer would agree with
- Metrics, gates and a baseline

157 unit tests and 32 golden cases. You need both, and they catch different things.

THE HARNESS · EVALUATION/HARNESS.PY

Three layers, from specific to general

Layer	Where	Example
Expectations	golden_cases.json	Loop → MANUAL_REVIEW, AGENT_LIMIT_REACHED, exactly 4 tool calls
Invariants	every case	Never approve a case that shouldn’t be approved
Gates	gates.json	100% pass, 0 unsafe approvals, p95 ≤ 5 s, tokens ≤ +20%

Expectations catch a wrong answer to one case. Invariants catch a wrong case. Gates decide whether you ship.

GOLDEN CASES · 32 IN FIVE FAMILIES

One case, one sentence of the spec

```
{"id": "G-T04",
"application": "APP-001",
"scenario": "Required fraud screen
returns 500",
"tool_fault": "fraud_screen=server_error",
"expect": {
"recommendation": "MANUAL_REVIEW",
"reasons_include": ["CHECK_UNAVAILABLE"]}}
```

baseline · 8 samples

llm · 6 model faults

agent · 5 loop and tool cases

tools · 7 check faults

workflow · 6 node faults

INVARIANTS · INVARIANT_VIOLATIONS()

Rules that hold on every case

No unsafe approval. Never approve what shouldn’t be approved.

No unfair decline. Never decline what policy doesn’t decline.

Approve needs a valid proposal. No approvals on a failed model.

Every decision has reasons. No empty reason codes.

The audit starts at intake. Nothing happens off the record.

Review means queued. MANUAL_REVIEW always reaches the queue.

ONE RUN · 32 CASES · ABOUT 2 SECONDS

Safety, quality, cost and speed in one report

0	100%	19%	1,881
unsafe approvals	cases pass	engine overrode the agent	input tokens per case

Also: escalation rate 62%, agent valid 75%, 1.06 tool calls per case, p95 79 ms. All on the mock, so the run is deterministic.

GATES · EVALS/GATES.JSON

A release is a set of numbers, not a feeling

Gate	Limit	Now
Pass rate	≥ 100%	100%
Unsafe approvals / unfair declines	0 / 0	0 / 0
Invariant violations	0	0
p95 latency	≤ 5,000 ms	79 ms
Mean input tokens	≤ 4,000	1,881
Tokens vs baseline	≤ +20%	+0.0%

The baseline (evals/baseline.json) is committed. Changing it is a reviewed diff with a reason.

MUTATION DRILLS

An eval that has never failed proves nothing

We broke	Cases failed	Gates failed
DTI review 43% → 47%	G-B02, G-B08	unsafe approvals, pass rate
Ignore required checks	G-T04	unsafe approvals, pass rate
Tool budget 4 → 40	G-A01	pass rate, input tokens
Patch MAX_TOOL_CALLS constant	none	none: it changed nothing

The last drill looked like a pass. The default argument was bound at import. That’s why the harness has its own tests.

LIVE WALKTHROUGH

Run it, break it, watch it go red

```
uv run underwriter eval
uv run underwriter eval --tag tools
uv run underwriter eval --markdown /tmp/eval.md
# edit decision/policy.py: dti_review_above = 0.47
uv run underwriter eval # FAILED
```

```
G-B02 MANUAL_REVIEW APPROVE FAIL: UNSAFE_APPROVAL: approved a case that should not be approved
FAIL unsafe approvals 1 ≤ 0
```

LAB · 100 MINUTES

Three exercises

30 MIN · CASES

Five new golden cases

Latency faults, two faults at once, the single agent with a bad number. Update the baseline and explain the change.

35 MIN · INVARIANT

Decline needs a reason

Every DECLINE carries a policy reason code. Forge a decision that breaks it, then check the real cases.

35 MIN · DRILLS

Two mutation drills

Make a harmless-looking change. If the eval stays green, write the case that would have caught it.

HOMEWORK AND INTERVIEW PREP

What strong work looks like

Criterion	Strong	Weak
Cases	A business sentence each	Output snapshots
Failures	Every fault family	Happy paths only
Invariants	Checked on every case	Scattered asserts
Gates	Safety, cost, latency	“Tests pass”
Trust	Drills prove it fails	Only ever green

Interview questions

- Evaluate a non-deterministic system
- Test vs eval: the difference
- Metrics that block a release
- Would your eval catch a regression?
- Keep eval cost and time down

TAKEAWAY

Measure the failures, not just the happy path, and prove your eval can fail.

Next · Lesson 7

Observability

OpenTelemetry traces for every node, tool and model call, with tokens and latency on each span.